want to interpolate an instance, class, or global variable, you can do so without the braces, just use #@var, #@@var, or #$var. However, the braces are recommended for consistency.

Strings can continue across multiple input lines, in which case they will contain newline characters.

You can use *here documents* to express long string literals. When Ruby parses the sequence <<IDENTIFIER or <<QUOTED_STRING, it replaces it with a string literal built from successive logical input lines. It stops building the string when it finds a line that starts with *identifier* or *quoted string*. You can put a minus sign immediately after the << characters, in which case the terminator can be indented from the margin. If a quoted string was used to specify the terminator, its quoting rules are applied to the here document; otherwise, double-quoting rules apply.

reference/quoting.rb

```
print <<HERE
Double quoted
here document.
It is #{Time.now}
HERE

print <<-'THERE'
    This is single quoted.
    The above used #{Time.now}
THERE
```

produces:

```
Double quoted
here document.
It is 2023-02-20 17:03:23 -0600
    This is single quoted.
    The above used #{Time.now}
```

Putting a tilde ~ in front of the identifier removes all leading spaces from each line of the here document, allowing the here doc to be indented for readability:

reference/heredoc.rb

```
print <<~HERE
  This is indented.
  But the result will not have
  the indentation.
HERE
```

produces:

```
This is indented.
But the result will not have
the indentation.
```

Adjacent single- and double-quoted strings are concatenated to form a single String object:

```
'Con' "cat" 'en' "ate" # => "Concatenate"
```

A new String object is created every time a string literal is assigned or passed as a parameter.

```
3.times do
  print "hello".object_id, " "
end
```